Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №6

“ Автоматизоване тестування Python-програм ”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи: сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

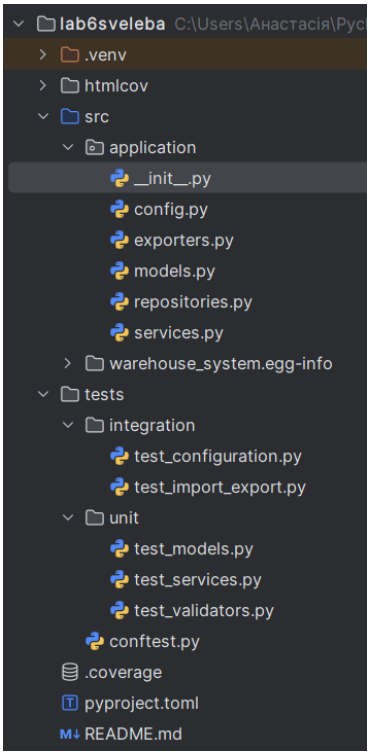
Варіант 10. Система обліку складських запасів

Протестувати:

- створення Product/WarehouseItem та валідацію quantity/price;
- операцію надходження;
- операцію списання;
- insufficient stock;
- пошук за кодом товару;
- розрахунок загальної вартості складу;
- визначення low-stock positions;
- boundary та invalid quantity/unit_price;
- CSV streaming import;
- malformed quantity record;
- JSON export;
- configuration low-stock threshold;
- WarehouseRepository через Mock/fake;
- supplier/reorder service через Mock із side_effect;
- async reorder provider через AsyncMock та integration stock flow.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src/application/ зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, README.md) та каталогу тестів tests/ із розділенням на unit/ та integration/.



2. У файлі pyproject.toml я налаштувала метадані пакета, необов'язкові залежності для розробки (pytest, pytest-cov, pytest-asyncio), параметри запуску тестувальника, а також правила перевірки покриття коду (coverage) із мінімальним порогом спрацювання не менше 80%.

```
[project]
name = "warehouse-system"
version = "0.1.0"
description = "Система обліку складських запасів (Варіант 10)"
requires-python = ">=3.10"
dependencies = []

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-cov>=5.0",
    "pytest-asyncio>=0.23.0",
]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = ["-ra", "--strict-markers"]
markers = [
    "integration: integration tests",
]

[tool.coverage.run]
branch = true
source = ["application"]

[tool.coverage.report]
show_missing = true
skip_covered = false
fail_under = 80
```

3. У файлі README.md я написала коротку документацію до проєкту та інструкцію однією командою для запуску тестів із генерацією звіту про покриття коду за допомогою pytest-cov.

````
# Система обліку складських запасів (Warehouse Management System)

Лабораторна робота №6 з дисципліни «Професійний Python»
**Варіант №10:** Система обліку складських запасів.

---

## Опис проєкту

Даний проєкт є системою автоматизованого тестування для доменної моделі та
сервісів системи обліку складських запасів. В рамках роботи реалізовано:
- Доменні моделі товарів (`WarehouseItem`) та контейнера складу (`Warehouse`) з
повною валідацією атрибутів.
- Сервіси бізнес-логіки для пошуку, розрахунку вартості інвентарю та визначення
позицій із низьким запасом.
- Універсальне сховище в пам'яті (`InMemoryRepository`) на базі Generics.
- Модулі імпорту та експорту даних у формати CSV та JSON (`CSVExporter`,
`CSVImporter`, `JSONExporter`).
- Асинхронний сервіс замовлення товарів (`AsyncReorderService`).
- Повний набір unit- та integration-тестів з використанням `pytest`, `fixtures`,
`tmp_path`, `monkeypatch`, `Mock` та `AsyncMock`.

---

## Структура проєкту

```text
warehouse_project/
├── pyproject.toml
├── README.md
├── src/
│   └── application/
│       ├── __init__.py          # Ініціалізація пакета та експорт компонентів
│       ├── models.py            # Доменні моделі (WarehouseItem, Warehouse)
│       ├── config.py            # Конфігурація застосунку та змінні оточення
│       ├── repositories.py      # Універсальне сховище InMemoryRepository
│       ├── services.py          # Бізнес-логіка та асинхронні сервіси
│       └── exporters.py         # Експорт/імспорт у CSV та JSON
└── tests/
    ├── conftest.py             # Спільні фікстури pytest (sample_item,
sample_warehouse)
    ├── unit/
    │   ├── test_models.py       # Модульні тести моделей
    │   ├── test_validators.py   # Тести валідації та boundary cases
(@pytest.mark.parametrize)
    │   └── test_services.py     # Тести сервісів, Mock, side_effect та AsyncMock
    └── integration/
        ├── test_configuration.py # Інтеграційні тести конфігурації
(monkeypatch)
        └── test_import_export.py # Інтеграційні тести імпорту/експорту
(tmp_path)
```

## Створення та активація віртуального середовища
```bash
python -m venv .venv
# Windows (PowerShell):
.\.venv\Scripts\Activate.ps1
# Linux/macOS:
source .venv/bin/activate
```
````

```
## Встановлення проекту та dev-залежностей
```bash
python -m pip install -e ".[dev]"
```

Запуск тестів та перевірка покриття
```bash
pytest -v --cov=application --cov-branch --cov-report term-missing --cov-
report=html
```
```

4. У файлі src/application/__init__.py я реалізувала ініціалізацію пакета застосунку та централізовано експортувала ключові доменні моделі (WarehouseItem, Warehouse), сервіси бізнес-логіки, репозиторій (InMemoryRepository), модулі експорту/імпорту (CSVExporter, CSVImporter, JSONExporter) та конфігураційні класи (AppConfig, get_config), забезпечивши єдину точку доступу до них.

```
from application.models import WarehouseItem, Warehouse
from application.services import (
    add_item,
    find_by_code,
    calculate_total_inventory_value,
    get_low_stock_items,
    AsyncReorderService,
)
from application.repositories import InMemoryRepository
from application.exporters import CSVExporter, CSVImporter, JSONExporter
from application.config import AppConfig, get_config

__all__ = [
    "WarehouseItem",
    "Warehouse",
    "add_item",
    "find_by_code",
    "calculate_total_inventory_value",
    "get_low_stock_items",
    "AsyncReorderService",
    "InMemoryRepository",
    "CSVExporter",
    "CSVImporter",
    "JSONExporter",
    "AppConfig",
    "get_config",
]
```

5. У файлі src/application/config.py я реалізувала зчитування налаштувань застосунку із змінних оточення (зокрема поріг низьких запасів LOW_STOCK_THRESHOLD та середовище виконання APP_ENV) за допомогою вбудованої бібліотеки os та датакласів.

```
import os
from dataclasses import dataclass

@dataclass
class AppConfig:
    low_stock_threshold: int
    environment: str

def get_config() -> AppConfig:
```

```
    threshold = int(os.getenv("LOW_STOCK_THRESHOLD", "5"))
    env = os.getenv("APP_ENV", "development")
    return AppConfig(low_stock_threshold=threshold, environment=env)
```

6. У файлі src/application/models.py я реалізувала доменні моделі предметної області: клас WarehouseItem із валідацією полів через __post_init__ (перевірка на порожній код/назву, заборона від'ємної кількості та ціни) й обчислювальною властивістю total_price, а також клас-контейнер Warehouse з підтримкою dunder-методів (__len__, __contains__).

```
from dataclasses import dataclass, field

@dataclass(frozen=True, slots=True)
class WarehouseItem:
    code: str
    name: str
    quantity: int
    unit_price: float

    def __post_init__(self) -> None:
        if not self.code.strip():
            raise ValueError("Product code cannot be empty")
        if not self.name.strip():
            raise ValueError("Product name cannot be empty")
        if self.quantity < 0:
            raise ValueError("Quantity cannot be negative")
        if self.unit_price < 0:
            raise ValueError("Unit price cannot be negative")

    @property
    def total_price(self) -> float:
        return self.quantity * self.unit_price

@dataclass
class Warehouse:
    items: dict[str, WarehouseItem] = field(default_factory=dict)

    def add(self, item: WarehouseItem) -> None:
        if item.code in self.items:
            existing = self.items[item.code]
            self.items[item.code] = WarehouseItem(
                code=existing.code,
                name=existing.name,
                quantity=existing.quantity + item.quantity,
                unit_price=existing.unit_price
            )
        else:
            self.items[item.code] = item

    def receive_stock(self, code: str, qty: int) -> None:
        if qty <= 0:
            raise ValueError("Received quantity must be positive")
        if code not in self.items:
            raise KeyError(f"Item with code {code} not found")
        item = self.items[code]
        self.items[code] = WarehouseItem(
            code=item.code,
            name=item.name,
            quantity=item.quantity + qty,
            unit_price=item.unit_price
        )

    def ship_stock(self, code: str, qty: int) -> None:
```

```
        if qty <= 0:
            raise ValueError("Shipped quantity must be positive")
        if code not in self.items:
            raise KeyError(f"Item with code {code} not found")
        item = self.items[code]
        if item.quantity < qty:
            raise ValueError("Insufficient stock for shipping")
        self.items[code] = WarehouseItem(
            code=item.code,
            name=item.name,
            quantity=item.quantity - qty,
            unit_price=item.unit_price
        )

    def __len__(self) -> int:
        return len(self.items)

    def __contains__(self, code: str) -> bool:
        return code in self.items
```

7. У файлі src/application/repositories.py я реалізувала універсальний клас сховища в оперативній пам'яті InMemoryRepository[T] із використанням дженериків (Generic та TypeVar) для зручного збереження, пошуку за ключем та отримання списку всіх доменних об'єктів.

```
from typing import Generic, TypeVar, Dict, List, Optional

T = TypeVar("T")

class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._storage: Dict[str, T] = {}

    def add(self, key: str, item: T) -> None:
        self._storage[key] = item

    def get_by_key(self, key: str) -> Optional[T]:
        return self._storage.get(key)

    def list_all(self) -> List[T]:
        return list(self._storage.values())
```

8. У файлі src/application/services.py я реалізувала бізнес-логіку керування складськими запасами (додавання товарів, пошук за кодом, розрахунок загальної вартості складу, фільтрація товарів із низьким запасом за порогом), а також асинхронний сервіс AsyncReorderService, який взаємодіє із зовнішнім API для автоматичного замовлення товарів.

```
import asyncio
from typing import List, Optional
from application.models import WarehouseItem

def add_item(warehouse_items: List[WarehouseItem], item: WarehouseItem) ->
List[WarehouseItem]:
    warehouse_items.append(item)
    return warehouse_items

def find_by_code(warehouse_items: List[WarehouseItem], code: str) ->
Optional[WarehouseItem]:
    for item in warehouse_items:
        if item.code == code:
```

```
            return item
    return None

def calculate_total_inventory_value(warehouse_items: List[WarehouseItem]) ->
float:
    return sum(item.total_price for item in warehouse_items)

def get_low_stock_items(warehouse_items: List[WarehouseItem], threshold: int) ->
List[WarehouseItem]:
    return [item for item in warehouse_items if item.quantity < threshold]

class AsyncReorderService:
    def __init__(self, async_client) -> None:
        self._client = async_client

    async def check_and_reorder_async(self, code: str, threshold: int,
current_qty: int) -> str:
        await asyncio.sleep(0.01)
        if current_qty < threshold:
            response = await self._client.place_order(code, threshold -
current_qty)
            return response.get("status", "pending")
        return "sufficient"
```

9. У файлі src/application/exporters.py я реалізувала класи CSVExporter, CSVImporter (для роботи з таблицями через csv.writer та csv.DictReader із перевіркою на пошкоджені записи malformed record) та JSONExporter для збереження й зчитування даних складських запасів у файлові формати.

```
import csv
import json
from pathlib import Path
from typing import List
from application.models import WarehouseItem

class CSVExporter:
    @staticmethod
    def export_to_file(items: List[WarehouseItem], file_path: Path) -> None:
        with open(file_path, mode="w", newline="", encoding="utf-8") as f:
            writer = csv.writer(f)
            writer.writerow(["code", "name", "quantity", "unit_price"])
            for i in items:
                writer.writerow([i.code, i.name, i.quantity, i.unit_price])

class CSVImporter:
    @staticmethod
    def import_from_file(file_path: Path) -> List[WarehouseItem]:
        items: List[WarehouseItem] = []
        with open(file_path, mode="r", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            for row in reader:
                if not all(k in row for k in ("code", "name", "quantity",
"unit_price")):
                    raise ValueError("Malformed CSV record: missing columns")
                # перевірка на валідність числових полів (malformed quantity
record)
                try:
                    qty = int(row["quantity"])
                    price = float(row["unit_price"])
                except ValueError as err:
                    raise ValueError("Malformed numeric record in CSV") from err

                item = WarehouseItem(
                    code=row["code"],
```

```
                    name=row["name"],
                    quantity=qty,
                    unit_price=price,
                )
                items.append(item)
        return items

class JSONExporter:
    @staticmethod
    def export_to_file(items: List[WarehouseItem], file_path: Path) -> None:
        data = [
            {"code": i.code, "name": i.name, "quantity": i.quantity,
"unit_price": i.unit_price}
            for i in items
        ]
        file_path.write_text(json.dumps(data, ensure_ascii=False, indent=2),
encoding="utf-8")
```

10. У файлі tests/conftest.py я реалізувала конфігурацію та фікстури pytest (створила готові тестові об'єкти sample_item, sample_item_list та sample_warehouse) для їх багаторазового й ізольованого використання в тестах.

```
import pytest
from application.models import WarehouseItem, Warehouse

@pytest.fixture
def sample_item() -> WarehouseItem:
    return WarehouseItem(code="P-001", name="Laptop", quantity=5,
unit_price=25000.0)

@pytest.fixture
def sample_item_list() -> list[WarehouseItem]:
    return [
        WarehouseItem(code="P-001", name="Laptop", quantity=5,
unit_price=25000.0),
        WarehouseItem(code="P-002", name="Mouse", quantity=2, unit_price=500.0),
        WarehouseItem(code="P-003", name="Keyboard", quantity=10,
unit_price=1200.0),
    ]

@pytest.fixture
def sample_warehouse(sample_item) -> Warehouse:
    wh = Warehouse()
    wh.add(sample_item)
    return wh
```

11. У файлі tests/unit/test_models.py я реалізувала модульні тести для перевірки правильності створення доменних моделей WarehouseItem та коректності роботи контейнера Warehouse із використанням стандартних тверджень assert.

```
import pytest
from application.models import WarehouseItem, Warehouse

def test_warehouse_item_creation(sample_item):
    assert sample_item.code == "P-001"
    assert sample_item.name == "Laptop"
    assert sample_item.total_price == 125000.0

def test_warehouse_container_operations(sample_warehouse, sample_item):
    assert len(sample_warehouse) == 1
```

```
    assert "P-001" in sample_warehouse
    assert "P-999" not in sample_warehouse

def test_receive_and_ship_stock(sample_item):
    wh = Warehouse()
    wh.add(sample_item)
    wh.receive_stock("P-001", 5)
    assert wh.items["P-001"].quantity == 10

    wh.ship_stock("P-001", 4)
    assert wh.items["P-001"].quantity == 6

def test_insufficient_stock_raises(sample_item):
    wh = Warehouse()
    wh.add(sample_item)
    with pytest.raises(ValueError, match="Insufficient stock"):
        wh.ship_stock("P-001", 20)

def test_invalid_stock_operations(sample_item):
    wh = Warehouse()
    wh.add(sample_item)
    with pytest.raises(ValueError, match="positive"):
        wh.receive_stock("P-001", 0)
    with pytest.raises(KeyError):
        wh.receive_stock("UNKNOWN", 5)
    with pytest.raises(ValueError, match="positive"):
        wh.ship_stock("P-001", -2)
    with pytest.raises(KeyError):
        wh.ship_stock("UNKNOWN", 5)
```

12. У файлі tests/unit/test_validators.py я реалізувала модульні тести для перевірки граничних значень (boundary testing) та викидання винятків ValueError через pytest.raises при передачі невалідних даних (від'ємні ціни, порожній код товару) з параметризацією @pytest.mark.parametrize.

```
import pytest
from application.models import WarehouseItem

@pytest.mark.parametrize("quantity", [0, 10, 100])
def test_valid_quantities(quantity):
    item = WarehouseItem(code="T-1", name="Test", quantity=quantity,
unit_price=10.0)
    assert item.quantity == quantity

@pytest.mark.parametrize("invalid_price", [-1.0, -100.5])
def test_invalid_price_raises_error(invalid_price):
    with pytest.raises(ValueError, match="negative"):
        WarehouseItem(code="T-1", name="Test", quantity=5,
unit_price=invalid_price)

def test_invalid_empty_code_raises_error():
    with pytest.raises(ValueError, match="code cannot be empty"):
        WarehouseItem(code="   ", name="Test", quantity=5, unit_price=10.0)

def test_invalid_empty_name_raises_error():
    with pytest.raises(ValueError, match="name cannot be empty"):
        WarehouseItem(code="T-1", name="   ", quantity=5, unit_price=10.0)
```

13. У файлі tests/unit/test_services.py я реалізувала модульні тести для перевірки бізнес-функцій сервісів, пошуку, розрахунків, а також тестування зовнішніх залежностей за допомогою Mock (перевірка

return_value, side_effect та interaction assertions типу assert_called_once_with) і асинхронних викликів за допомогою AsyncMock і pytest.mark.asyncio.

```
import pytest
from unittest.mock import Mock, AsyncMock
from application.services import (
    find_by_code,
    calculate_total_inventory_value,
    get_low_stock_items,
    AsyncReorderService,
)


def test_find_by_code(sample_item_list):
    found = find_by_code(sample_item_list, "P-002")
    assert found is not None
    assert found.name == "Mouse"
    assert find_by_code(sample_item_list, "UNKNOWN") is None


def test_calculate_total_inventory_value(sample_item_list):
    total = calculate_total_inventory_value(sample_item_list)
    assert total == pytest.approx(138000.0)


def test_get_low_stock_items(sample_item_list):
    low = get_low_stock_items(sample_item_list, threshold=6)
    assert len(low) == 2
    assert low[0].code == "P-001"
    assert low[1].code == "P-002"


def test_mock_supplier_notification():
    mock_supplier = Mock()
    mock_supplier.reorder.return_value = "ORD-999"

    result = mock_supplier.reorder("P-001", 10)
    assert result == "ORD-999"
    mock_supplier.reorder.assert_called_once_with("P-001", 10)


def test_mock_supplier_side_effect():
    mock_supplier = Mock()
    mock_supplier.reorder.side_effect = ConnectionError("Supplier timeout")

    with pytest.raises(ConnectionError, match="Supplier timeout"):
        mock_supplier.reorder("P-001", 10)


@pytest.mark.asyncio
async def test_async_reorder_service():
    mock_client = AsyncMock()
    mock_client.place_order.return_value = {"status": "success"}

    service = AsyncReorderService(async_client=mock_client)
    result = await service.check_and_reorder_async("P-001", 10, 3)

    assert result == "success"
    mock_client.place_order.assert_awaited_once_with("P-001", 7)


@pytest.mark.asyncio
async def test_async_reorder_service_sufficient():
    mock_client = AsyncMock()
```

```
    service = AsyncReorderService(async_client=mock_client)
    result = await service.check_and_reorder_async("P-001", 5, 10)

    assert result == "sufficient"
    mock_client.place_order.assert_not_awaited()
```

14. У файлі tests/integration/test_configuration.py я реалізувала інтеграційний тест для перевірки зчитування конфігурації застосунку з використанням фікстури monkeypatch для безпечної та тимчасової зміни змінних середовища.

```
from application.config import get_config


def test_configuration_loading(monkeypatch):
    monkeypatch.setenv("LOW_STOCK_THRESHOLD", "15")
    monkeypatch.setenv("APP_ENV", "testing")

    config = get_config()
    assert config.low_stock_threshold == 15
    assert config.environment == "testing"


def test_configuration_defaults(monkeypatch):
    monkeypatch.delenv("LOW_STOCK_THRESHOLD", raising=False)
    monkeypatch.delenv("APP_ENV", raising=False)

    config = get_config()
    assert config.low_stock_threshold == 5
    assert config.environment == "development"
```

15. У файлі tests/integration/test_import_export.py я реалізувала інтеграційні тести файлових операцій (імпорт/експорт CSV та JSON) із використанням фікстури tmp_path для повної ізоляції файлової системи, а також перевірку обробки помилок при завантаженні пошкоджених CSV-файлів.

```
import pytest
from application.exporters import CSVExporter, CSVImporter, JSONExporter


@pytest.mark.integration
def test_csv_export_and_import_integration(tmp_path, sample_item_list):
    file_path = tmp_path / "warehouse.csv"
    CSVExporter.export_to_file(sample_item_list, file_path)
    assert file_path.exists()

    imported = CSVImporter.import_from_file(file_path)
    assert len(imported) == 3
    assert imported[0].code == sample item list[0].code
    assert imported[0].unit_price ==
pytest.approx(sample_item_list[0].unit_price)


@pytest.mark.integration
def test_json_export_integration(tmp_path, sample_item_list):
    json_path = tmp_path / "warehouse.json"
    JSONExporter.export_to_file(sample_item_list, json_path)
    assert json_path.exists()

    content = json_path.read_text(encoding="utf-8")
    assert "Laptop" in content
    assert "P-003" in content
```

```python
@pytest.mark.integration
def test_import_malformed_csv_raises(tmp_path):
    bad_csv = tmp_path / "bad.csv"
    bad_csv.write_text("code,name,quantity\nP-1,Test,10\n", encoding="utf-8")

    with pytest.raises(ValueError, match="Malformed CSV record"):
        CSVImporter.import_from_file(bad_csv)


@pytest.mark.integration
def test_import_malformed_numeric_raises(tmp_path):
    bad_csv = tmp_path / "bad_num.csv"
    bad_csv.write_text("code,name,quantity,unit_price\nP-1,Test,abc,10.0\n",
encoding="utf-8")

    with pytest.raises(ValueError, match="Malformed numeric record"):
        CSVImporter.import_from_file(bad_csv)
```

Результати:

Встановлення залежностей проєкту:

(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> python -m pip install -e ".[dev]"

Obtaining file:///C:/Users/%D0%90%D0%BD%D0%B0%D1%81%D1%82%D0%B0%D1%81%D1%96%D1%8F/PycharmProjects/lab6sveleba

Installing build dependencies ... done

Checking if build backend supports build_editable ... done

Getting requirements to build editable ... done

Preparing editable metadata (pyproject.toml) ... done

Requirement already satisfied: pytest>=8.0 in .\.venv\Lib\site-packages (from warehouse-system==0.1.0) (9.1.1)

Requirement already satisfied: pytest-cov>=5.0 in .\.venv\Lib\site-packages (from warehouse-system==0.1.0) (7.1.0)

Requirement already satisfied: pytest-asyncio>=0.23.0 in .\.venv\Lib\site-packages (from warehouse-system==0.1.0) (1.4.0)

Requirement already satisfied: colorama>=0.4 in .\.venv\Lib\site-packages (from pytest>=8.0->warehouse-system==0.1.0) (0.4.6)

Requirement already satisfied: iniconfig>=1.0.1 in .\.venv\Lib\site-packages (from pytest>=8.0->warehouse-system==0.1.0) (2.3.1)

Requirement already satisfied: packaging>=22 in .\.venv\Lib\site-packages (from pytest>=8.0->warehouse-system==0.1.0) (26.3)

Requirement already satisfied: pluggy<2,>=1.5 in .\.venv\Lib\site-packages (from pytest>=8.0->warehouse-system==0.1.0) (1.7.0)

Requirement already satisfied: pygments>=2.7.2 in .\.venv\Lib\site-packages (from pytest>=8.0->warehouse-system==0.1.0) (2.21.0)

Requirement already satisfied: coverage>=7.10.6 in .\.venv\Lib\site-packages (from coverage[toml]>=7.10.6->pytest-cov>=5.0->warehouse-system==0.1.0) (7.16.2)

Building wheels for collected packages: warehouse-system

Building editable for warehouse-system (pyproject.toml) ... done

Created wheel for warehouse-system: filename=warehouse_system-0.1.0-0.editable-py3-none-any.whl size=1461 sha256=c5c947dd77d48db9c3318372dfb1ce801c7ac76df04aea675540875ab211a5c8

Stored in directory: C:\Users\Анастасія\AppData\Local\Temp\pip-ephem-wheel-cache-9jesyr6t\wheels\d0\97\66\747ef14fd073bb7503331511d2061741eb032ce136ca749e52

Successfully built warehouse-system

Installing collected packages: warehouse-system

Attempting uninstall: warehouse-system

Found existing installation: warehouse-system 0.1.0

Uninstalling warehouse-system-0.1.0:

Successfully uninstalled warehouse-system-0.1.0

Successfully installed warehouse-system-0.1.0

[notice] A new release of pip is available: 26.1.2 -> 26.2.1

[notice] To update, run: python.exe -m pip install --upgrade pip

Запуск тестів та генерація HTML-звіту покриття:

(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6sveleba> pytest -v --cov=application --cov-branch --cov-report term-missing --cov-report=html

============================= test session starts ==============================

platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\Анастасія\PycharmProjects\lab6sveleba\.venv\Scripts\python.exe

cachedir: .pytest_cache

rootdir: C:\Users\Анастасія\PycharmProjects\lab6sveleba

configfile: pyproject.toml

testpaths: tests

plugins: asyncio-1.4.0, cov-7.1.0

asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function

collected 25 items

tests/integration/test_configuration.py::test_configuration_loading PASSED [4%]

tests/integration/test_configuration.py::test_configuration_defaults PASSED [8%]

tests/integration/test_import_export.py::test_csv_export_and_import_integration PASSED [12%]

tests/integration/test_import_export.py::test_json_export_integration PASSED [16%]

tests/integration/test_import_export.py::test_import_malformed_csv_raises PASSED [20%]

tests/integration/test_import_export.py::test_import_malformed_numeric_raises PASSED [24%]

tests/unit/test_models.py::test_warehouse_item_creation PASSED [28%]

tests/unit/test_models.py::test_warehouse_container_operations PASSED [32%]

tests/unit/test_models.py::test_receive_and_ship_stock PASSED [36%]

tests/unit/test_models.py::test_insufficient_stock_raises PASSED [40%]

tests/unit/test_models.py::test_invalid_stock_operations PASSED [44%]

tests/unit/test_services.py::test_find_by_code PASSED [48%]

tests/unit/test_services.py::test_calculate_total_inventory_value PASSED [52%]

tests/unit/test_services.py::test_get_low_stock_items PASSED [56%]

tests/unit/test_services.py::test_mock_supplier_notification PASSED [60%]

tests/unit/test_services.py::test_mock_supplier_side_effect PASSED [64%]

tests/unit/test_services.py::test_async_reorder_service PASSED [68%]

tests/unit/test_services.py::test_async_reorder_service_sufficient PASSED [72%]

tests/unit/test_validators.py::test_valid_quantities[0] PASSED [76%]

tests/unit/test_validators.py::test_valid_quantities[10] PASSED [80%]

tests/unit/test_validators.py::test_valid_quantities[100] PASSED [84%]

tests/unit/test_validators.py::test_invalid_price_raises_error[-1.0] PASSED [88%]

tests/unit/test_validators.py::test_invalid_price_raises_error[-100.5] PASSED [92%]

tests/unit/test_validators.py::test_invalid_empty_code_raises_error PASSED [96%]

tests/unit/test_validators.py::test_invalid_empty_name_raises_error PASSED [100%]

========== tests coverage ==========

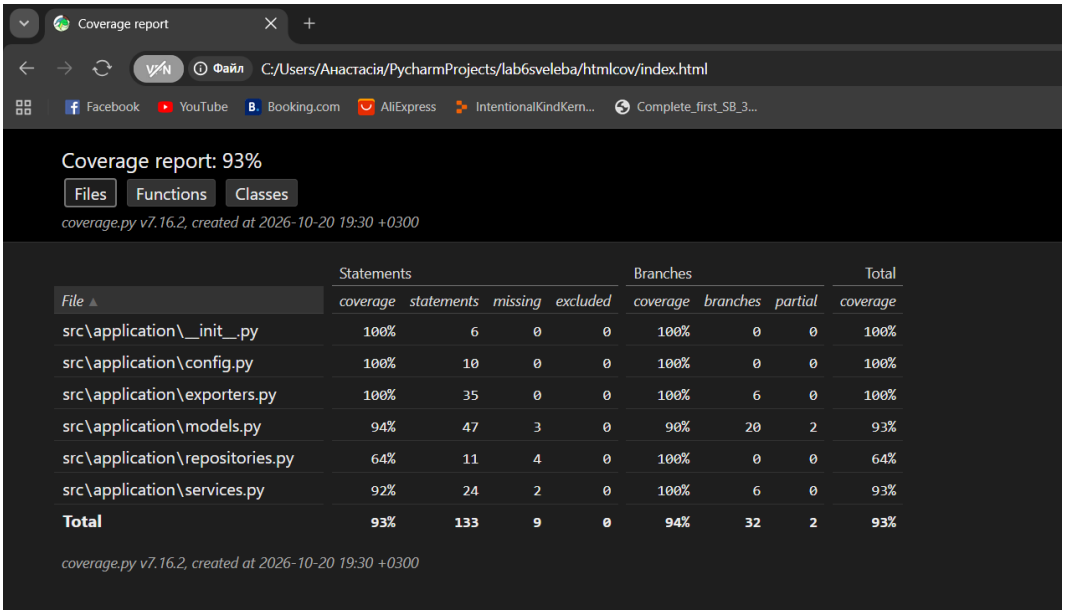
_____ coverage: platform win32, python 3.13.14-final-0 _____

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src\application__init__.py	6	0	0	0	100%	
src\application\config.py	10	0	0	0	100%	
src\application\exporters.py	35	0	6	0	100%	
src\application\models.py	47	3	20	2	93%	16, 30-31
src\application\repositories.py	11	4	0	0	64%	7, 10, 13, 16
src\application\services.py	24	2	6	0	93%	6-7
TOTAL	133	9	32	2	93%	

Coverage HTML written to dir htmlcov

Required test coverage of 80.0% reached. Total coverage: 93.33%

========== 25 passed in 0.38s ==========



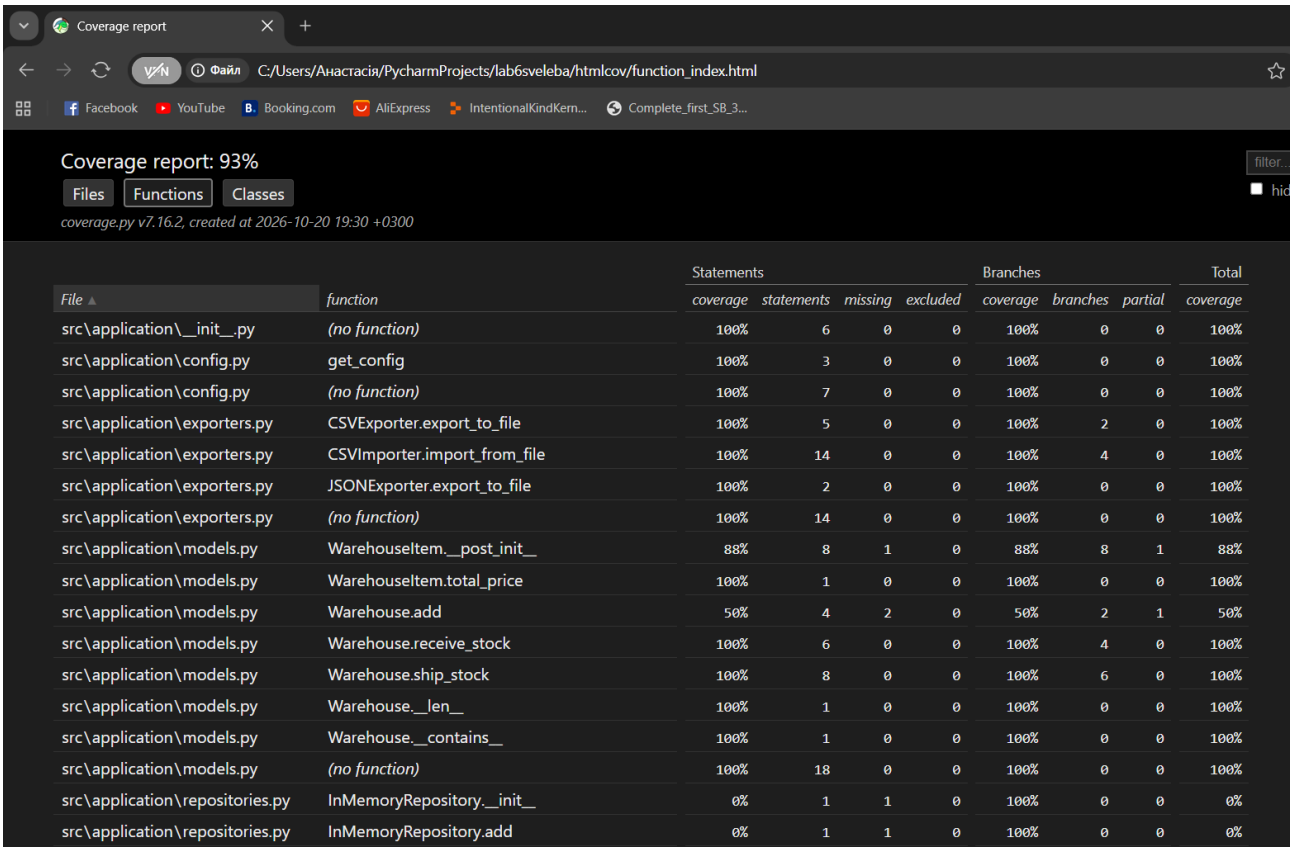
Coverage report: 93%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-20 19:30 +0300

	Statements				Branches			Total
File	*coverage*	*statements*	*missing*	*excluded*	*coverage*	*branches*	*partial*	*coverage*
src\application__init__.py	100%	6	0	0	100%	0	0	100%
src\application\config.py	100%	10	0	0	100%	0	0	100%
src\application\exporters.py	100%	35	0	0	100%	6	0	100%
src\application\models.py	94%	47	3	0	90%	20	2	93%
src\application\repositories.py	64%	11	4	0	100%	0	0	64%
src\application\services.py	92%	24	2	0	100%	6	0	93%
Total	**93%**	**133**	**9**	**0**	**94%**	**32**	**2**	**93%**

coverage.py v7.16.2, created at 2026-10-20 19:30 +0300



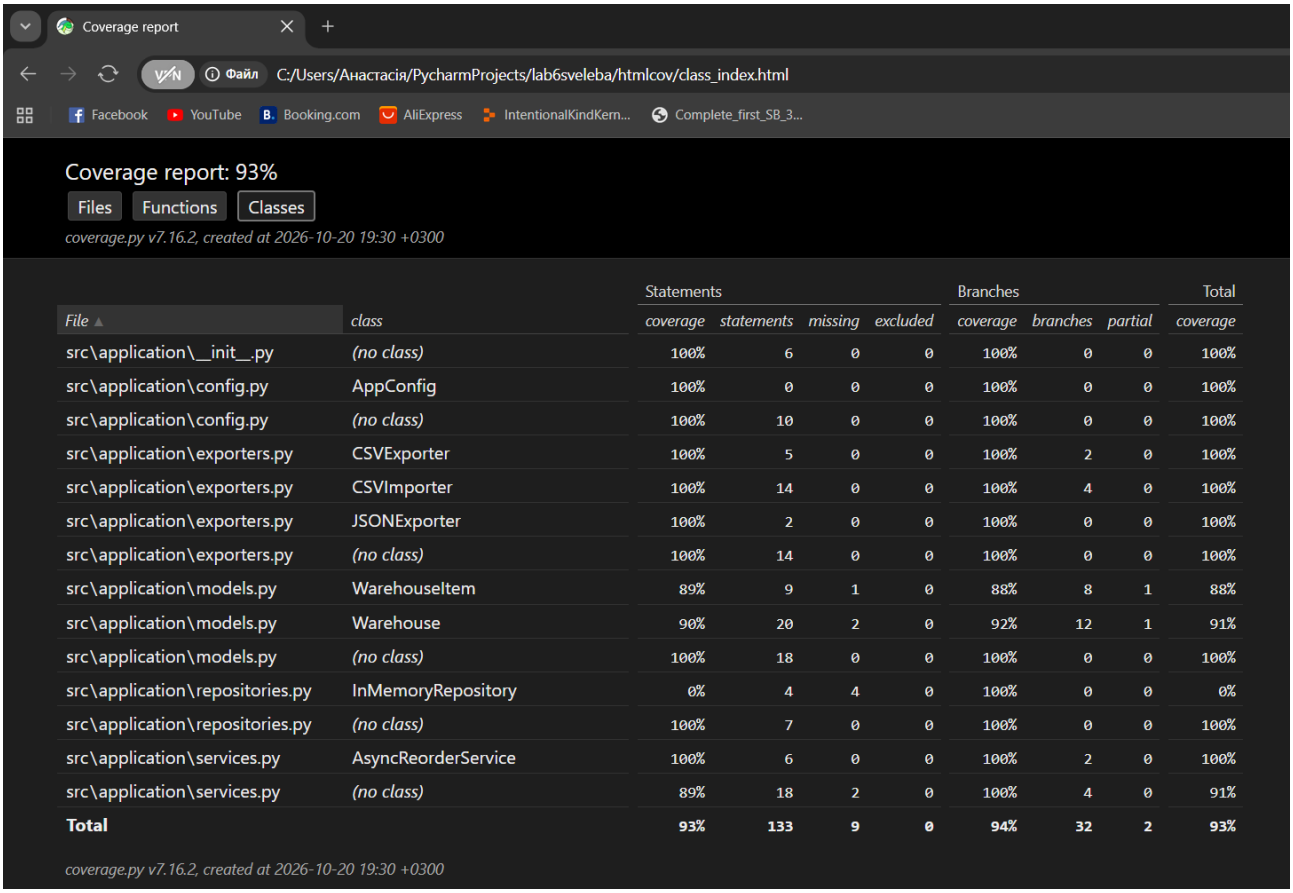
Coverage report: 93%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-20 19:30 +0300

		Statements				Branches			Total
File	*function*	*coverage*	*statements*	*missing*	*excluded*	*coverage*	*branches*	*partial*	*coverage*
src\application__init__.py	*(no function)*	100%	6	0	0	100%	0	0	100%
src\application\config.py	get_config	100%	3	0	0	100%	0	0	100%
src\application\config.py	*(no function)*	100%	7	0	0	100%	0	0	100%
src\application\exporters.py	CSVExporter.export_to_file	100%	5	0	0	100%	2	0	100%
src\application\exporters.py	CSVImporter.import_from_file	100%	14	0	0	100%	4	0	100%
src\application\exporters.py	JSONExporter.export_to_file	100%	2	0	0	100%	0	0	100%
src\application\exporters.py	*(no function)*	100%	14	0	0	100%	0	0	100%
src\application\models.py	WarehouseItem.__post_init__	88%	8	1	0	88%	8	1	88%
src\application\models.py	WarehouseItem.total_price	100%	1	0	0	100%	0	0	100%
src\application\models.py	Warehouse.add	50%	4	2	0	50%	2	1	50%
src\application\models.py	Warehouse.receive_stock	100%	6	0	0	100%	4	0	100%
src\application\models.py	Warehouse.ship_stock	100%	8	0	0	100%	6	0	100%
src\application\models.py	Warehouse.__len__	100%	1	0	0	100%	0	0	100%
src\application\models.py	Warehouse.__contains__	100%	1	0	0	100%	0	0	100%
src\application\models.py	*(no function)*	100%	18	0	0	100%	0	0	100%
src\application\repositories.py	InMemoryRepository.__init__	0%	1	1	0	100%	0	0	0%
src\application\repositories.py	InMemoryRepository.add	0%	1	1	0	100%	0	0	0%
src\application\repositories.py	InMemoryRepository.get_by_key	0%	1	1	0	100%	0	0	0%
src\application\repositories.py	InMemoryRepository.list_all	0%	1	1	0	100%	0	0	0%
src\application\repositories.py	*(no function)*	100%	7	0	0	100%	0	0	100%
src\application\services.py	add_item	0%	2	2	0	100%	0	0	0%
src\application\services.py	find_by_code	100%	4	0	0	100%	4	0	100%
src\application\services.py	calculate_total_inventory_value	100%	1	0	0	100%	0	0	100%
src\application\services.py	get_low_stock_items	100%	1	0	0	100%	0	0	100%
src\application\services.py	AsyncReorderService.__init__	100%	1	0	0	100%	0	0	100%
src\application\services.py	AsyncReorderService.check_and_reorder_async	100%	5	0	0	100%	2	0	100%
src\application\services.py	*(no function)*	100%	10	0	0	100%	0	0	100%
Total		**93%**	**133**	**9**	**0**	**94%**	**32**	**2**	**93%**

coverage.py v7.16.2, created at 2026-10-20 19:30 +0300



		Statements				Branches			Total
File ▲	class	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\application__init__.py	(no class)	100%	6	0	0	100%	0	0	100%
src\application\config.py	AppConfig	100%	0	0	0	100%	0	0	100%
src\application\config.py	(no class)	100%	10	0	0	100%	0	0	100%
src\application\exporters.py	CSVExporter	100%	5	0	0	100%	2	0	100%
src\application\exporters.py	CSVImporter	100%	14	0	0	100%	4	0	100%
src\application\exporters.py	JSONExporter	100%	2	0	0	100%	0	0	100%
src\application\exporters.py	(no class)	100%	14	0	0	100%	0	0	100%
src\application\models.py	WarehouseItem	89%	9	1	0	88%	8	1	88%
src\application\models.py	Warehouse	90%	20	2	0	92%	12	1	91%
src\application\models.py	(no class)	100%	18	0	0	100%	0	0	100%
src\application\repositories.py	InMemoryRepository	0%	4	4	0	100%	0	0	0%
src\application\repositories.py	(no class)	100%	7	0	0	100%	0	0	100%
src\application\services.py	AsyncReorderService	100%	6	0	0	100%	2	0	100%
src\application\services.py	(no class)	89%	18	2	0	100%	4	0	91%
Total		**93%**	**133**	**9**	**0**	**94%**	**32**	**2**	**93%**

Висновок: у ході виконання лабораторної роботи я розробила систему керування складськими запасами мовою Python, яка забезпечує реєстрацію товарів, облік їхньої кількості, надходження та списання продукції, обчислення загальної вартості запасів і визначення товарів із низьким залишком. Також я реалізувала імпорт даних із CSV-файлів, експорт інформації у формат JSON, налаштування параметрів системи через змінні середовища та асинхронну взаємодію з постачальником. Для перевірки правильності роботи програми було створено модульні й інтеграційні тести з використанням pytest, фікстур, mock-об’єктів і засобів тестування асинхронного коду. У процесі тестування перевірено коректність роботи основних компонентів системи, обробку некоректних даних і виникнення помилкових ситуацій. У результаті виконання роботи я отримала практичні навички організації Python-проєкту, написання автоматизованих тестів, перевірки взаємодії між компонентами та налаштування контролю покриття коду тестами.